

XSS in GMAIL Dynamic Email (AMP for Email)

XSS in GMAIL Dynamic Email (AMP for Email) - asdqw3, Medium.

- Published: 2026-07-07
- Original: <<https://asdqw3.medium.com/xss-in-gmail-dynamic-email-amp-for-email-3872d6052a0d>>
- Preserved from: <https://asdqw3.medium.com/xss-in-gmail-dynamic-email-amp-for-email-3872d6052a0d> (live) on 2026-08-10
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Bug Bounty

Bug Bounty Writeup

Vulnerability

Google

Xss Attack

XSS in GMAIL Dynamic Email (AMP for Email)

[[image: asdqw3]](https://asdqw3.medium.com/?source=post_page---byline-3872d6052a0d-----)

[asdqw3](#)

I found a XSS vulnerability on GMAIL that I reported to Google VRP on January 2023. This issue occurs due to improper HTML parsing in GMAIL Dynamic email (AMP for Email).

بسم الله الرحمن الرحيم

AMP for Email

AMP for email allows senders to include AMP components inside rich engaging emails, making modern app functionality available within email. The AMP email format provides a

subset of AMPHTML components for use in email messages, that allows recipients of AMP emails to interact dynamically with content directly in the message. (<https://amp.dev/about/email>)

How does it work?

An AMP email message MUST

- start with the doctype `<!doctype html>`.
- contain a top-level `<html >4email>` tag (`<html amp4email>` is accepted as well).
- contain `<head>` and `<body>` tags (They are optional in HTML).
- contain a `<meta charset="utf-8">` tag as the first child of their head tag.
- contain a `<script async src="https://cdn.ampproject.org/v0.js"></script>` tag inside their head tag.
- contain amp4email boilerplate (`<style amp4email-boilerplate>body{visibility:hidden}</style>`) inside their head tag to initially hide the content until AMP JS is loaded.

Example valid AMP for Email message

Specifying CSS in an AMP document

All CSS in any AMP document must be included in a `<style amp-custom>` tag within the header or as inline `style` attributes.

Custom CSS in an AMP for Email document

Discovery

As far as I know, there are two XSS vulnerabilities in GMAIL AMP which were publicly disclosed, one of them was discovered by Michał Bentkowski, you can read the writeup here <https://research.securitum.com/xss-in-amp4email-dom-clobbering/> and the other one was discovered by Adi “Adico” Cohen, you can read the writeup here <https://www.adico.me/post/xss-in-gmail-s-amp4email>, after reading both of the writeups multiple times, I decided to give a try to explore GMAIL AMP through their **playground**, in the hope of finding a bypass or new XSS vector.

My first attempt was trying Adico’s payload and check what is the HTML parser do after the fix. Adico managed to find the XSS by injecting `</style>` closing tag into the CSS selector by encoding the letter **y** `*to*\000079`

source: <https://www.adico.me/post/xss-in-gmail-s-amp4email>

When it sent to GMAIL, `\000079` was decoded back to letter **y**, in the result it turn to a valid `</style>` close tag, then break the `<style amp-custom>` tag and add `<img>` element to the document `<body>`.

source: <https://www.adico.me/post/xss-in-gmail-s-amp4email>

Then, I test with following payload:

It parsed into:

“`<>`” characters inside a string were encoded to `\00003c` & `\00003e`

\000069 decoded to letter i, **but \00003c & \00003e not decoded to back to "<" & ">"**

Also noticed that **div>span** was fine, > **character does not encoded to *\00003e*, so my assumption regarding Google's fix was that they only encoded <> characters if the character present in string between " " or ' ', make sense since the "greater than" sign (>) in the css selector is a valid symbol which used as element to element selector.

Then I tried sending "<>" characters in different locations until I found a promising spot. We are allowed to put any characters into a CSS rule set right after the **property:value** declaration.

source: <https://www.thecodesmith.co/css/css-rulesets>

For example, we are allowed to write any text or HTML tag like following:

As expected, **</style>** closing tag is not allowed.

Again, I tried multiple html tag combinations, then I found following snippets that surprising me when it parsed in GMAIL:

When it sent to GMAIL, it's parsed as follow:

Seems like the parser still parse the **</style** even if it doesn't have a closing bracket ****>**.
****Also**, noticed that the parser auto generated closing tag for each html tag, so what if we include <style> tag? will the parser generate the closing tag too?

The answer is YES!

Then I quickly tried basic XSS payload, however nothing appears in the body element. It seems like they added another filter to prevent the XSS.

I tried every single html tags, no one works but <meta>. I was able to inject <meta> tag with http-equiv = refresh.

Final payload:

```
<style amp-custom>style>a{font-family:'asdqwe'</style</head><body>
<style/> <meta http-equiv="refresh" content="10;url=data:text/html,
<h1>HELLO!!</h1><script>alert()</script>"/></style>
```

After 10 seconds

Unfortunately, there are strict CSP rules in place on GMAIL, so the XSS not executed. Tried few times to find the bypass but no luck.

I found this bug in January 2023 and immediately report it to Google VRP and awarded a bounty of \$6000 (\$5000 + \$1000 bonus).

Thanks